LeetCode Practice

10 Easy: String

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

10
EASY

Patterns (in order)

1. String (10)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

10 questions | tap a title to jump to it

Easy questions (10) Go >

String (10) Go >

E 1. Length of Last Word LC 58
E 2. Isomorphic Strings LC 205
E 3. Word Pattern LC 290
E 4. Ransom Note LC 383
E 5. First Unique Character in a String LC 387
E 6. Repeated Substring Pattern LC 459
E 7. Sentence Similarity LC 734
E 8. Rotate String LC 796
E 9. Single Row Keyboard LC 1165
E 10. Maximum Number of Balloons LC 1189

Easy questions (10)

EASY | TOPIC 1 OF 1

String

10 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Length of Last Word LC 58
2. Isomorphic Strings LC 205
3. Word Pattern LC 290
4. Ransom Note LC 383
5. First Unique Character in a String LC 387
6. Repeated Substring Pattern LC 459
7. Sentence Similarity LC 734
8. Rotate String LC 796
9. Single Row Keyboard LC 1165
10. Maximum Number of Balloons LC 1189

EASY | String | LeetCode #58 | 1 of 10 | 1/10

Length of Last Word

QUESTION

Given a string s consisting of words and spaces, return the length of the last word in the string.

A word is a maximal substring consisting of non-space characters only.

Example 1	
Input	`s = "Hello World"`
Output	5
Explanation	*The last word is "World" with length 5.*

Example 2	
Input	`s = " fly me to the moon "`
Output	4
Explanation	*The last word is "moon" with length 4.*

Example 3	
Input	`s = "luffy is still joyboy"`
Output	6
Explanation	*The last word is "joyboy" with length 6.*

CONSTRAINTS

- 1 <= s.length <= 10^4
- s consists of only English letters and spaces ' '.
- There will be at least one word in s.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the length of the last word in s. Words are separated by spaces.

Trailing spaces should be ignored. Right?"

"A few quick questions:

Guaranteed at least one word? Yes. Can have leading spaces? Yes — ignore all."

"Let me walk: s='Hello World' -> len('World')=5 ok. s=' fly me to the moon ' -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
split the string on spaces
return the length of the last word
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n)	O(n)

NOTE

Not slow in time; split makes a copy of every word. Walking back from the end (skip spaces, then count letters) is O(n) worst case with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Scan from the right — O(n). Walk-through:"

`i = len(s) - 1.`

Skip trailing spaces: while i>=0 and s[i]==' ': i--.

Count the last word: while i>=0 and s[i]!=' ': i--; length++.

Return length.

WHY scan from right: the last word ends at the last non-space character.

Scanning from right lets us skip trailing spaces first, then count directly.

Alternative: split(). s.split()[-1] gives the last word; return len of it.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _58_LengthOfLastWord:
    def lengthOfLastWord(self, s):
        i = len(s) - 1
        while i >= 0 and s[i] == ' ':
            i -= 1
        length = 0
        while i >= 0 and s[i] != ' ':
            length += 1
            i -= 1
        return length
```

PHASE 5 TEST & DEBUG

"Let me trace lengthOfLastWord('Hello World')."

Strip trailing spaces: 'Hello World'. No trailing spaces here.

Find last space: index 5. Length = 11 - 5 - 1 = 5 ('World').

Or iterate from end: i=10('d'),9('l'),8('r'),7('o'),6('W'),5(' ')->stop. Length=5 ok

'Hello World ': strip->'Hello World'. Last word='World', len=5 ok

'a': only one word -> length=1 ok. 'Hello': length=5 ok.

WHY strip then split: trailing spaces would make last 'word' empty -> wrong count.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
at most one full scan	only an integer counter

The split() approach is 2 lines and perfectly clean — mention it as the readable alternative.

Follow-up: count total words — use split() or count transitions from space to non-space.

Follow-up: First and Last Word Length — extend to scan from both ends.

IN YOUR APP | Expose | Editorial | Community

EASY | String | LeetCode #205 | 2 of 10 | 2/10

Isomorphic Strings

QUESTION

Given two strings s and t, determine if they are isomorphic.

Two strings s and t are isomorphic if the characters in s can be replaced to get t.

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1

Input	`s = "egg", t = "add"`
Output	`true`
Explanation	*The strings s and t can be made identical by:*

Example 2

Input	`s = "f11", t = "b23"`
Output	`false`
Explanation	*The strings s and t can not be made identical as '1' needs to be mapped to both '2' and '3'.*

Example 3

Input	`s = "paper", t = "title"`
Output	`true`

CONSTRAINTS

- 1 <= s.length <= 5 * 10^4
- t.length == s.length
- s and t consist of any valid ascii character.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Two strings s and t are isomorphic if there's a one-to-one character mapping from s to t.

Every character in s maps to exactly one character in t and vice versa. Right?"

"A few quick questions:

Same length? Yes — guaranteed. Case-sensitive? Yes."

"Let me walk: s='egg', t='add' -> e->a, g->d -> valid. s='foo', t='bar' -> o->a and o->r conflict -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (i, j) in pairs(positions):
    s[i] == s[j] must hold exactly when t[i] == t[j]
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Compares every pair of positions. Two hash maps (or arrays of 256) recording each letter's partner check consistency in one pass: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Bijection map in both directions — O(n). Walk-through:"

s_to_t={}, t_to_s={}. For each (sc, tc) in zip(s, t):

If sc in s_to_t and s_to_t[sc] != tc: return False. <- sc maps to different tc

If tc in t_to_s and t_to_s[tc] != sc: return False. <- tc maps from different sc

s_to_t[sc]=tc. t_to_s[tc]=sc.

Return True.

WHY two maps: isomorphism requires a bijection — one-to-one AND onto.

One map catches 'egg'->'add' (e->a, g->d: valid).

Two maps prevent 'ab'->'aa' (b would also map to a, but a already maps from a, not b).

WHY check both before setting: detect conflicts immediately.

Time: O(n). Space: O(|charset|).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _205_IsomorphicStrings:
    def isIsomorphic(self, s, t):
        s_map, t_map = {}, {}
        for cs, ct in zip(s, t):
            if s_map.get(cs, ct) != ct or t_map.get(ct, cs
) != cs:
                return False
            s_map[cs] = ct
            t_map[ct] = cs
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace isIsomorphic('egg','add')."

s='egg', t='add'. Two maps: s->t and t->s.

s[0]='e' -> t[0]='a'. s_map={e:a}. t_map={a:e}. ok

s[1]='g' -> t[1]='d'. s_map={e:a,g:d}. t_map={a:e,d:g}. ok

s[2]='g' -> t[2]='d'. s_map[g]=d==d ok. t_map[d]=g==g ok. Match.

Return True ok

'foo','bar': s_map={f:b,o:a}. At s[2]='o': s_map[o]=a!=r -> False ok

'badc','baba': s_map={b:b,a:a,d:b}: d->b but b already->b? t_map[b]: first b->b, second b->a -> conflict. False ok

Trace: s="egg", t="add" -> map e->a ok, map g->d ok, map g->d (already mapped) ok -> True. s="foo", t="bar" -> map f->b, map o->a, map o->r (conflict: o already->a) -> False.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** bounded alphabet

The dict.get(key, default) pattern is clean — returns default if key missing.

Follow-up: Word Pattern (#290) — same bijection idea but between words and characters.

Follow-up: Encode String with Shortest Length — inverse of isomorphism.

EASY | String | LeetCode #290 | 3 of 10 | 3/10

Word Pattern

QUESTION

Given a pattern and a string s, find if s follows the same pattern.

Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in s. Specifically:

CONSTRAINTS

- Each letter in pattern maps to exactly one unique word in s.
- Each unique word in s maps to exactly one letter in pattern.
- No two letters map to the same word, and no two words map to the same letter.

Example 1	
Input	`pattern = "abba", s = "dog cat cat dog"`
Output	`true`
Explanation	*The bijection can be established as:*

Example 2	
Input	`pattern = "abba", s = "dog cat cat fish"`
Output	`false`

Example 3	
Input	`pattern = "aaaa", s = "dog cat cat dog"`
Output	`false`

- 1 <= pattern.length <= 300
- pattern contains only lower-case English letters.
- 1 <= s.length <= 3000
- s contains only lowercase English letters and spaces ' '.
- s does not contain any leading or trailing spaces.
- All the words in s are separated by a single space.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

The pattern string maps each character to a word in s.

Both mappings must be bijective — no two chars map to same word and vice versa. Right?"

"A few quick questions:

Number of words in s must equal pattern length? Yes — otherwise false.

Case-sensitive? Yes."

"Let me walk: pattern='abba', s='dog cat cat dog' -> a->dog, b->cat -> valid ok

PHASE 1 CLARIFY

pattern='abba', s='dog cat cat fish' -> a maps to both dog and fish -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (i, j) in pairs(positions):
    pattern[i] == pattern[j] must hold exactly
        when word[i] == word[j]
```

TIME	SPACE
O(n^2 * L)	O(1)

WHY IT IS SLOW

Compares every pair and compares whole words. Two hash maps (letter to word and word to letter) check the bijection in one pass: O(total length).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Bijection map word->pattern char — O(n). Walk-through:"

`words = s.split(). If len(words) != len(pattern): return False.`

w2p={}, p2w={}. For (word, pc) in zip(words, pattern):

If word in w2p and w2p[word] != pc: return False.

If pc in p2w and p2w[pc] != word: return False.

w2p[word]=pc. p2w[pc]=word.

Return True.

WHY two maps: prevent 'aabb' from matching 'dog dog' (would pass with one map since a->dog, b->dog,

but p2w catches that d is already mapped from 'dog', and now b also maps to 'dog' -> conflict on p2w).

Equivalent to Q205 (Isomorphic Strings) but on words.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _290_WordPattern:
    def wordPattern(self, pattern, s):
        words = s.split()
        if len(pattern) != len(words):
            return False
        char_to_word, word_to_char = {}, {}
        for char, word in zip(pattern, words):
            if char_to_word.get(char, word) != word:
                return False
            if word_to_char.get(word, char) != char:
                return False
            char_to_word[char] = word
            word_to_char[word] = char
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the word-pattern mapping — show where a conflict first arises."

pattern='abba', words=['dog','cat','cat','dog']:

a,dog: ok. b,cat: ok. b,cat: ok (already mapped). a,dog: ok -> True ok

pattern='abba', words=['dog','cat','cat','fish']:

a->dog. b->cat. b->cat ok. a: char_to_word[a]=dog != fish -> False ok

pattern='aaaa', words=['dog','dog','dog','dog']: all consistent -> True ok

pattern='aabb', words=['dog','dog','cat','cat']: a->dog, then b->cat -> True ok

Checking both directions prevents aa->ab (char same but words differ)

Trace: pattern="abba", s="dog cat cat dog" -> map a->dog, b->cat, b->cat ok, a->dog ok -> True. pattern="abba", s="dog cat cat fish" -> a->dog, b->cat, b->cat ok, a->fish != dog -> False.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
one pass	up to 26+n entries

This is Isomorphic Strings (#205) at the word level — same bijection pattern.

Follow-up: Isomorphic Strings (#205) — char-level version of this exact problem.

Follow-up: if pattern can use wildcards, add regex matching.

IN YOUR APP | Expose | Editorial | Community

EASY | String | LeetCode #383 | 4 of 10 | 4/10

Ransom Note

QUESTION

Given two strings ransomNote and magazine, return true if ransomNote can be constructed by using the letters from magazine and false otherwise.

Each letter in magazine can only be used once in ransomNote.

Example 1	
Input	`ransomNote = "a", magazine = "b"`
Output	`false`

Example 2	
Input	`ransomNote = "aa", magazine = "ab"`
Output	`false`

Example 3	
Input	`ransomNote = "aa", magazine = "aab"`
Output	`true`

CONSTRAINTS

- 1 <= ransomNote.length, magazine.length <= 10^5
- ransomNote and magazine consist of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We need to check if every character in ransomNote can be supplied

by magazine — one letter from magazine per letter in ransomNote.

So if ransomNote needs two 'a's, magazine must have at least two 'a's. Right?"

"A few quick questions:

Are both strings guaranteed to be non-empty? Yes, length >= 1.

Only lowercase English letters — so exactly 26 possible characters. Good.

We're not modifying the strings, just checking feasibility?"

"Let me walk the examples:

ransomNote='a', magazine='b': 'a' not in magazine -> false ok

ransomNote='aa', magazine='ab': need two 'a's, magazine has one -> false ok

ransomNote='aa', magazine='aab': need two 'a's, magazine has two -> true ok Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for letter in ransom_note:
    search the magazine for an unused copy of
        that letter and mark it used
```

TIME	SPACE
O(n * m)	O(m)

WHY IT IS SLOW

Rescans the magazine for every letter. Count the magazine's letters (26 slots) and decrement per ransom letter: O(n + m) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is scanning magazine for each character — that's the n*m.

The question we keep asking is: how many of this letter does magazine still have?

A frequency counter answers that in O(1).

I'll count every character in magazine upfront with a Counter.

Then walk ransomNote: for each character, decrement its count.

If any count drops below zero, magazine doesn't have enough of that letter — return false.

If I finish ransomNote without going negative, return true.

Pseudocode:

```
freq = Counter(magazine)
for char in ransomNote:
freq[char] -= 1
if freq[char] < 0: return False
return True
```

Time: O(n + m) — one pass each over magazine and ransomNote.

Space: O(1) — the Counter holds at most 26 keys (lowercase letters only).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _383_RansomNote:
    def canConstruct(self, ransomNote: str, magazine: str)
 -> bool:
        available = Counter(magazine)
        for char in ransomNote:
            available[char] -= 1
            if available[char] < 0:
                return False
```

```
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the frequency counts for magazine and ransom note and show the deficit check."

"Let me trace through the three examples."

ransomNote='a', magazine='b'

`available = {'b': 1}`

char='a': available['a']=0-1=-1 < 0 -> return False ok

(Counter returns 0 for missing keys, so no KeyError)

ransomNote='aa', magazine='ab'

`available = {'a':1, 'b':1}`

char='a': available['a']=0 -> ok

char='a': available['a']=-1 < 0 -> return False ok

ransomNote='aa', magazine='aab'

`available = {'a':2, 'b':1}`

char='a': available['a']=1 -> ok

char='a': available['a']=0 -> ok

`return True ok`

Edge — ransomNote longer than magazine: ransomNote='abc', magazine='ab'

`available = {'a':1, 'b':1}`

char='a': 0 -> ok. char='b': 0 -> ok. char='c': -1 < 0 -> return False ok

Counter's default of 0 for missing keys handles unseen characters cleanly

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n + m)	**O(1)**
O(m) to build the Counter, O(n) to walk ransomNote	the Counter has at most 26 entries since only lowercase letters exist

Even though we write O(k) where k is distinct chars, k is bounded by 26 — constant.

Alternative: Counter subtraction in one line.

`return not (Counter(ransomNote) - Counter(magazine))`

Counter subtraction drops keys with zero or negative counts,

so if anything remains, ransomNote needs letters magazine doesn't have.

Elegant but less readable in an interview — I'd use the explicit loop.

Another alternative: sort both strings and use two pointers — O(n log n + m log m),

slower but uses O(1) space beyond the sort. Worth mentioning if interviewer

constraints disallow hash maps.

PHASE 6 COMPLEXITY & FOLLOW-UP

Given more time I'd ask: what if characters include uppercase or unicode?

We'd replace the 26-slot assumption with a full hash map — space becomes O(alphabet size).

IN YOUR APP | Expose | Editorial | Community

EASY | String | LeetCode #387 | 5 of 10 | 5/10

First Unique Character in a String

QUESTION

Given a string s, find the first non-repeating character in it and return its index. If it does not exist, return -1.

Example 1	
Input	`s = "leetcode"`
Output	`0`
Explanation	*The character 'l' at index 0 is the first character that does not occur at any other index.*

Example 2	
Input	`s = "loveleetcode"`
Output	`2`

Example 3	
Input	`s = "aabb"`
Output	`-1`

CONSTRAINTS

- $1 <= s.length <= 10^5$
- s consists of only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the index of the first non-repeating character in s. Return -1 if none. Right?"

"A few quick questions:

Only lowercase English letters? Yes. Can all characters repeat? Yes — return -1."

"Let me walk: s='leetcode' -> 'l' at index 0 (l:1,e:3,t:1,c:1,o:1,d:1) -> 0 ok

s='loveleetcode' -> 'v' at index 2 -> 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    count how many times s[i] occurs by scanning
        the string
return the first index whose count is 1
```

TIME	SPACE
O(n^2)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

A scan per character. Count all letters once, then scan again for the first with count 1: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) count() call inside an O(n) loop.

Pre-count all frequencies in one pass with Counter.

Second pass: return the first index where count == 1.

Time: O(n). Space: O(1) — at most 26 entries."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _387_FirstUniqueChar:
    def firstUniqChar(self, s):
        from collections import Counter
        count = Counter(s)
        for i, ch in enumerate(s):
            if count[ch] == 1:
                return i
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace firstUniqChar on s='leetcode'."

count: l:1,e:3,t:1,c:1,o:1,d:1

Scan left to right: l->count[l]=1 -> return index 0 ok

Trace s='loveleetcode': l:2,o:2,v:1,e:4,t:1,c:1,d:1 -> scan: l(2 skip),o(2 skip),v(1)->return index 2 ok

WHY two-pass: first build counts O(n), then scan for first count==1 O(n).

Alternative: one dict preserving insertion order; O(n) either way.

Edge: s='aabb' -> all count>=2 -> return -1.

Result: 0 (first char 'l' has count=1 in 'leetcode'). Order matters—return leftmost unique.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)
two passes	fixed 26-entry Counter

Follow-up: streaming input — use OrderedDict or a queue to maintain insertion order.

Follow-up: First Unique Number in a Stream (#1429) — same idea for integers.

IN YOUR APP | Expose | Editorial | Community

EASY | String | LeetCode #459 | 6 of 10 | 6/10

Repeated Substring Pattern

QUESTION

Given a string s, check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1	
Input	`s = "abab"`
Output	`true`
Explanation	*It is the substring "ab" twice.*

Example 2	
Input	`s = "aba"`
Output	`false`

Example 3	
Input	`s = "abcabcabcabc"`
Output	`true`
Explanation	*It is the substring "abc" four times or the substring "abcabc" twice.*

CONSTRAINTS

- $1 <= s.length <= 10^4$
- s consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if s can be formed by repeating a shorter substring multiple times. Right?"

"A few quick questions:

At least 2 repetitions? Yes. Empty string -> false? Yes per constraints."

"Let me walk: s='abab' -> 'ab' repeated 2x -> true ok. s='aba' -> no valid substring -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for l in ls:
    # that divides n
    check whether s is its first L characters
        repeated
return True if one works
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n * d(n))	O(n)

WHY IT IS SLOW

Rebuilds and compares the string for every divisor. s is a repeated pattern exactly when s appears inside (s + s) with the first and last character removed: O(n) with KMP or a linear search.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"KMP failure function: if (n % (n - fail[-1])) == 0, repeated pattern found — O(n). Walk-through:"

Compute KMP failure function (lps): lps[i] = length of longest proper prefix = suffix of s[:i+1].

`period = n - lps[-1]. (smallest repeating unit length)`

If n % period == 0: s consists of that repeating unit -> return True.

WHY failure function: the last value lps[-1] tells us the longest proper prefix that's also a suffix.

If s is a repeated pattern: lps[-1] = n - period (the pattern length).

WHY n % period == 0: the pattern divides evenly into s.

Simpler check: (s+s)[1:-1].find(s) != -1. (O(n) but less obvious why)

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _459_RepeatedSubstringPattern:
    def repeatedSubstringPattern(self, s):
        doubled = s + s
        return s in doubled[1:-1]
```

PHASE 5 TEST & DEBUG

"Let me trace the KMP failure function on s+'#'+s and show why a period divides len(s)."

s='abab': doubled='abababab'. doubled[1:-1]='bababab'. 'abab' in 'bababab'->True ok

s='aba': doubled='abaaba'. [1:-1]='baab'. 'aba' in 'baab'->False ok

s='abcabcabcabc': doubled[1:-1] contains 'abcabcabcabc' -> True ok

Slicing [1:-1] avoids the trivial case where s is found at position 0 or n

Trace: root=[1,2,3] -> left subtree (2) checked then right (3); both children of root compared.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
Python's 'in' uses efficient string matching	for doubled string

KMP failure function approach: if failure[-1] != 0 and len(s) % (len(s)-failure[-1]) == 0.

Follow-up: find the shortest repeating unit — check smallest divisor length.

Follow-up: Rotate String (#796) — same doubled-string trick.

IN YOUR APP | Expose | Editorial | Community

EASY | String | LeetCode #734 | 7 of 10 | 7/10

Sentence Similarity

QUESTION

We can represent a sentence as an array of words, for example, the sentence "I am happy with leetcode" can be represented as arr = ["I","am",happy","with","leetcode"].

Given two sentences sentence1 and sentence2 each represented as a string array and given an array of string pairs similarPairs where similarPairs[i] = [x_i, y_i] indicates that the two words x_i and y_i are similar.

Return true if sentence1 and sentence2 are similar, or false if they are not similar.

Two sentences are similar if:

CONSTRAINTS

- They have the same length (i.e., the same number of words)
- sentence1[i] and sentence2[i] are similar.

Notice that a word is always similar to itself, also notice that the similarity relation is not transitive. For example, if the words a and b are similar, and the words b and c are similar, a and c are not necessarily similar.

Example 1

Input	`sentence1 = ["great","acting","skills"], sentence2 = ["fine","drama","talent"], similarPairs = [["great","fine"],["drama","acting"],["skills","talent"]]`
Output	`true`
Explanation	*The two sentences have the same length and each word i of sentence1 is also similar to the corresponding word in sentence2.*

Example 2

Input	`sentence1 = ["great"], sentence2 = ["great"], similarPairs = []`
Output	`true`
Explanation	*A word is similar to itself.*

Example 3

Input	`sentence1 = ["great"], sentence2 = ["doubleplus","good"], similarPairs = [["great","doubleplus"]]`
Output	`false`
Explanation	*As they don't have the same length, we return false.*

- 1 <= sentence1.length, sentence2.length <= 1000
- 1 <= sentence1[i].length, sentence2[i].length <= 20
- sentence1[i] and sentence2[i] consist of English letters.

QUESTION

- 0 <= similarPairs.length <= 1000
- similarPairs[i].length == 2
- 1 <= x_i.length, y_i.length <= 20
- x_i and y_i consist of lower-case and upper-case English letters.
- All the pairs (x_i,y_i) are distinct.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Two sentences are similar if they're the same length and every word at position i

in sentence1 is similar to the word at position i in sentence2.

A word is always similar to itself.

Similarity is NOT transitive — if a~b and b~c, that doesn't mean a~c.

We're given the list of similar pairs explicitly, so we only trust those. Right?"

"A few quick questions:

Can similarPairs be empty? Yes — then only identical words are similar.

Are pairs directional? The problem implies symmetric — if [a,b] is given,

then a~b AND b~a. I'll confirm: yes, similarity is symmetric.

Can the same pair appear twice? Constraints say all pairs are distinct — good."

"Let me walk example 1:

sentence1=['great','acting','skills'], sentence2=['fine','drama','talent'].

Same length: 3. pairs include [great,fine],[drama,acting],[skills,talent].

great~fine ok, acting~drama ok (symmetric), skills~talent ok -> true ok Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for b) in b)s:
    scan all similarity pairs to see if they
        match (either order)
return False on a mismatch
```

TIME	SPACE
O(n * p)	O(1)

WHY IT IS SLOW

A scan of all p pairs per word. Put the similar pairs in a hash set (both orders) and test each word pair in O(1): O(n + p).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is scanning all p pairs for every word position.

We're asking the same question repeatedly: is word2 similar to word1?

A hash map from each word to its set of similar words answers that in O(1).

Preprocess similarPairs once: for each [a, b], add b to hmap[a] and a to hmap[b].

That handles symmetry automatically — no need to check both directions at lookup time.

Then for each (word1, word2) pair in the sentences:

`if they're equal, they're trivially similar — skip.`

Otherwise check if word2 is in hmap[word1].

Using defaultdict(set) means missing keys return an empty set — no KeyError.

Pseudocode:

```
if len(s1) != len(s2): return False
similar = defaultdict(set)
for a, b in pairs: similar[a].add(b);
  similar[b].add(a)
for w1, w2 in zip(s1, s2):
if w1 != w2 and w2 not in similar[w1]: return False
return True
```

Time: O(p + n). Space: O(p) for the map. Does that make sense?"

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _734_SentenceSimilarity:
    def areSentencesSimilar(self, sentence1: List[str], se
ntence2: List[str], similarPairs: List[List[str]]) -> bool
:
        if len(sentence1) != len(sentence2):
            return False

        similar = defaultdict(set)
        for word_a, word_b in similarPairs:
            similar[word_a].add(word_b)
            similar[word_b].add(word_a)       # symmetric:
 both directions

        for word1, word2 in zip(sentence1, sentence2):
            if word1 != word2 and word2 not in similar[wor
d1]:
                return False

        return True
```

PHASE 5 TEST & DEBUG

"Let me trace each word pair against the synonym set and show when a mismatch is caught."

"Let me trace through all three examples."

Example 1: s1=['great','acting','skills'], s2=['fine','drama','talent']

pairs=[['great','fine'],['drama','acting'],['skills','talent']]

```
similar = {great:{fine}, fine:{great}, drama:{acting},
  acting:{drama}, skills:{talent}, talent:{skills}}
```

(great,fine): great!=fine, fine in similar[great] ok

(acting,drama): acting!=drama, drama in similar[acting] ok

(skills,talent): skills!=talent, talent in similar[skills] ok

`return True ok`

Example 2: s1=['great'], s2=['great'], pairs=[]

Lengths match. similar={} (empty).

(great,great): great==great -> skip.

`return True ok`

Example 3: s1=['great'], s2=['doubleplus','good'], pairs=[['great','doubleplus']]

`len(s1)=1 != len(s2)=2 -> return False immediately ok`

Non-transitive check: pairs=[['a','b'],['b','c']], checking a~c

`similar = {a:{b}, b:{a,c}, c:{b}}`

word1='a', word2='c': a!=c, c not in similar[a]={b} -> return False ok

Confirms we don't accidentally apply transitivity.

All cases pass including the transitivity trap

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(p + n)	**O(p)**
O(p) to build the similarity map, O(n) to check each word pair	for the map — each pair adds two entries

The critical design decision is storing both directions upfront rather than

checking both at lookup time. It keeps the lookup a single O(1) set membership test.

Important constraint to call out: similarity is NOT transitive.

This rules out Union-Find (which is designed for transitive equivalence classes).

If transitivity WERE required — for example in Sentence Similarity II (#737) —

we'd use Union-Find to group words into connected components and check

`if sentence1[i] and sentence2[i] share the same root.`

Given more time I'd ask: what if the same word pair could appear in different cases,

like ['Great','fine'] vs ['great','fine']? We'd normalize to lowercase before building

the map and before lookups.

IN YOUR APP | Expose | Editorial | Community

EASY | String | LeetCode #796 | 8 of 10 | 8/10

Rotate String

QUESTION

Given two strings s and goal, return true if and only if s can become goal after some number of shifts on s.

A shift on s consists of moving the leftmost character of s to the rightmost position.

CONSTRAINTS

- For example, if s = "abcde", then it will be "bcdea" after one shift.

Example 1	
Input	`s = "abcde", goal = "cdeab"`
Output	`true`

Example 2	
Input	`s = "abcde", goal = "abced"`
Output	`false`

- 1 <= s.length, goal.length <= 100
- s and goal consist of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if we can rotate s (shift left any number of times) to get goal. Right?"

"A few quick questions:

Same length required? Yes. Empty strings? Both empty -> true."

"Let me walk: s='abcde', goal='cdeab' -> rotate left by 2 -> 'cdeab' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for rotation in s:
    if it equals goal:
        return True
return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Builds and compares n rotations. If the lengths match, goal is a rotation of s exactly when it is a substring of s + s: O(n) with KMP.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is the O(n) comparison for each of n rotations.

Key insight: any rotation of s is a substring of s+s.

So just check if goal is a substring of s+s (and lengths match).

Time: O(n) with built-in find. Space: O(n) for s+s.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _796_RotateString:
    def rotateString(self, s, goal):
        return len(s) == len(goal) and goal in s + s
```

PHASE 5 TEST & DEBUG

"Let me trace s+s and check if goal appears as a substring — show the concatenation and the search."

s='abcde', goal='cdeab': s+s='abcdeabcde'. 'cdeab' in 'abcdeabcde' ok -> True ok

s='abcde', goal='abced': 'abced' not in 'abcdeabcde' -> False ok

s='aa', goal='aa': s+s='aaaa'. 'aa' in 'aaaa' ok -> True ok

The length check guards against goal being a substring of s+s but with different length

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) Python's 'in' uses Boyer-Moore-Horspool internally	O(n) creates s+s as a new string

This is the classic 'all rotations as substrings' trick — worth naming explicitly.

Follow-up: String Rotation (CTCI 1.9) — same problem, different phrasing.

Follow-up: if multiple rotation checks needed, use KMP for O(n) guaranteed.

EASY | String | LeetCode #1165 | 9 of 10 | 9/10

Single Row Keyboard

QUESTION

There is a special keyboard with all keys in a single row.

Given a string keyboard of length 26 indicating the layout of the keyboard (indexed from 0 to 25). Initially, your finger is at index 0. To type a character, you have to move your finger to the index of the desired character. The time taken to move your finger from index i to index j is |i - j|.

You want to type a string word. Write a function to calculate how much time it takes to type it with one finger.

Example 1

Input	`keyboard = "abcdefghijklmnopqrstuvwxyz", word = "cba"`
Output	`4`
Explanation	*The index moves from 0 to 2 to write 'c' then to 1 to write 'b' then to 0 again to write 'a'. Total time = 2 + 1 + 1 = 4.*

Example 2

Input	`keyboard = "pqrstuvwxyzabcdefghijklmno", word = "leetcode"`
Output	`73`

CONSTRAINTS

- keyboard.length == 26
- keyboard contains each English lowercase letter exactly once in some order.
- 1 <= word.length <= 10^4
- word[i] is an English lowercase letter.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We have a custom keyboard layout — 26 characters in some arbitrary order.

Our finger starts at index 0. To type each character in word,

we move from the current index to that character's index on the keyboard.

The cost is the absolute distance moved. We sum all moves and return the total. Right?"

"A few quick questions:

The keyboard always has exactly 26 letters, each exactly once — so it's a permutation.

Word can only contain lowercase letters, and every letter in word is guaranteed

to appear on the keyboard — so no missing-key edge case to handle.

Word length can be up to 10^4 — so O(n) should be fine."

"Let me walk example 1:

keyboard='abcdefghijklmnopqrstuvwxyz', word='cba'.

Start at 0. 'c' is at index 2: |0-2|=2. 'b' is at 1: |2-1|=1. 'a' is at 0: |1-0|=1.

Total = 2+1+1 = 4. Got it."

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for letter in word:
    find its position in the keyboard string by
        scanning
    add the distance from the previous position
```

TIME	SPACE
O(n * 26)	O(1)

WHY IT IS SLOW

A scan of up to 26 letters per character. A map from letter to key index (built once) makes each lookup O(1): O(n + 26).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The brute force scans 26 characters for every letter typed — that's the inner O(26).

We're asking the same question repeatedly: where does this letter live on the keyboard?

A hash map answers that in O(1) after a one-time O(26) build.

Preprocess keyboard once: build a map from character -> index.

Then walk word: for each character, look up its index in O(1),

add the absolute distance from the current position, update position, move on.

Pseudocode:

```
pos_map = {char: i for i, char in enumerate(keyboard)}
```

total, position = 0, 0

```
for char in word:
total += abs(position - pos_map[char])
position = pos_map[char]
return total
```

Time: O(26 + n) = O(n). Space: O(26) = O(1) — the map is fixed size.

PHASE 3 OPTIMIZE

In practice the constant factor improvement is small since 26 is tiny,

but the hash map is the correct interview answer — cleaner and scales to

larger alphabets.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1165_SingleRowKeyboard:
    def calculateTime(self, keyboard: str, word: str) -> i
nt:
        key_index = {char: i for i, char in enumerate(keyb
oard)}
        total = 0
        position = 0
        for char in word:
            total += abs(position - key_index[char])
            position = key_index[char]
        return total
```

PHASE 5 TEST & DEBUG

"Let me trace each character of word against the keyboard position array and accumulate distance."

"Let me trace through both examples."

keyboard='abcdefghijklmnopqrstuvwxyz', word='cba'

`key_index = {'a':0,'b':1,'c':2,...}`

char='c': |0-2|=2, position=2. total=2

char='b': |2-1|=1, position=1. total=3

char='a': |1-0|=1, position=0. total=4

`return 4 ok`

keyboard='pqrstuvwxyzabcdefghijklmno', word='leetcode'

`key_index = {'p':0,'q':1,...,'a':10,'b':11,...,'l':21, 'e':14,'t':19,'c':12,'o':24,'d':13}`

char='l': |0-21|=21, position=21. total=21

char='e': |21-14|=7, position=14. total=28

char='e': |14-14|=0, position=14. total=28

char='t': |14-19|=5, position=19. total=33

char='c': |19-12|=7, position=12. total=40

char='o': |12-24|=12, position=24. total=52

char='d': |24-13|=11, position=13. total=63

char='e': |13-14|=1, position=14. total=64 <- wait, let me recheck key_index

(keyboard='pqrstuvwxyzabcdefghijklmno': p=0,q=1,r=2,s=3,t=4,u=5,v=6,w=7,x=8,y=9,z=10,

a=11,b=12,c=13,d=14,e=15,f=16,g=17,h=18,i=19,j=20,k=21,l=22, m=23,n=24,o=25)

char='l': |0-22|=22. char='e': |22-15|=7. char='e': 0. char='t': |15-4|=11.

char='c': |4-13|=9. char='o': |13-25|=12. char='d': |25-14|=11. char='e': |14-15|=1.

PHASE 5 TEST & DEBUG

`total = 22+7+0+11+9+12+11+1 = 73 ok`

Manual trace confirms both examples

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
O(26) to build the map, O(n) to walk word	26-entry map

The brute force using keyboard.index() is also O(n) in practice since keyboard is

always 26 characters — but the hash map is the canonical interview answer because

it makes the O(1) lookup intent explicit and generalizes to any alphabet size.

Trade-off: if keyboard could be very long (say a 10^6-char layout), the hash map

preprocessing cost would matter more — but we'd still prefer it over .index().

One-liner alternative:

```
pos = {c: i for i, c in enumerate(keyboard)}
return sum(abs(pos[b] - pos[a]) for a, b in zip('_' + word, word))
```

where prepending '_' shifts word so each pair is (prev_char, curr_char),

and '_' maps to nothing — so you'd actually prepend a dummy start:

`pairs = zip([None] + list(word), word) — but this gets messy.`

The explicit loop is cleaner in an interview.

Given more time I'd ask: what if we can use two fingers?

That becomes a DP problem — optimal two-finger typing is a classic hard variant.

EASY | String | LeetCode #1189 | 10 of 10 | 10/10

Maximum Number of Balloons

QUESTION

Given a string text, you want to use the characters of text to form as many instances of the word "balloon" as possible.

You can use each character in text at most once. Return the maximum number of instances that can be formed.

Example 1	
Input	`text = "nlaebolko"`
Output	`1`

Example 2	
Input	`text = "loonbalxballpoon"`
Output	`2`

Example 3	
Input	`text = "leetcode"`
Output	`0`

CONSTRAINTS

- 1 <= text.length <= 10^4
- text consists of lower case English letters only.
- Note: This question is the same as 2287: Rearrange Characters to Make Target String.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the maximum number of times 'balloon' can be formed from the characters in text.

Each letter in text can only be used once. Right?"

"A few quick questions:

'balloon' has b:1, a:1, l:2, o:2, n:1 — note l and o appear twice.

Only lowercase English letters in text? Yes."

"Let me walk: text='nlaebolko' -> can form 'balloon' once ok

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count each letter of the text
the answer is the minimum of count(b), count(a),
    count(l) / 2, count(o) / 2, count(n)
```

TIME	SPACE
O(n)	O(1)

PHASE 2 BRUTE FORCE (also optimal)

NOTE

Not slow: one pass over the text and a constant-size count table; this is also the optimal approach.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Frequency count + floor division — O(n). Walk-through:"

Count frequency of each character in text.

'balloon' requires: b×1, a×1, l×2, o×2, n×1.

`occurrences = min(`

freq['b'], freq['a'], freq['l']//2, freq['o']//2, freq['n']

).

WHY floor division for l and o: 'balloon' needs 2 of each.

How many complete 'balloon' words fit = minimum supply across all required characters,

with l and o halved since each word needs 2.

General approach for any target word: freq[c]//need[c] for each c in target, take min.

Time: O(n). Space: O(1) (26-char alphabet).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1189_MaximumBalloons:
    def maxNumberOfBalloons(self, text):
        from collections import Counter
        freq = Counter(text)
        return min(
            freq['b'],
            freq['a'],
            freq['l'] // 2,
            freq['o'] // 2,
            freq['n'],
        )
```

PHASE 5 TEST & DEBUG

"Let me trace the frequency counts for 'balloon' letters and show which letter limits the answer."

text='nlaebolko': freq={n:1,l:2,a:1,e:1,b:1,o:2,k:1}.

`min(b:1, a:1, l:1, o:1, n:1) = 1 ok`

text='loonbalxballpoon': l:4,o:4,n:2,b:2,a:2,... -> min(2,2,2,2,2)=2 ok

text='': freq empty. min(0,0,0,0,0)=0 ok

Integer division for l and o handles the 'twice needed' requirement

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** Counter holds at most 26 chars

The key insight: l and o appear TWICE in 'balloon' so we must floor-divide by 2.

Follow-up: Ransom Note (#383) — same idea, check if magazine covers the note.

Follow-up: if we could use each char up to k times, multiply each available count by k.

IN YOUR APP | Expose | Editorial | Community